

Dans le cadre de ce TP, l'objectif était de mettre en place un service DNS fonctionnel, accompagné d'un serveur web hébergé localement.

Le travail consistait à configurer un serveur BIND9 assurant la résolution directe et inverse d'un domaine spécifique que nous devons choisir et qui faisait autorité sur la zone ac-monge.fr. Nous devons aussi mettre en place un forwarder pour la résolution des domaines externes.

Un serveur Apache2 a également été installé afin d'héberger une page web accessible via le nom de domaine défini.

Enfin, un mécanisme de blocage DNS a été ajouté pour interdire la résolution de certains sites (comme leboncoin.fr et amazon.uk) .

SOMMAIRE :

1- Mise en place du DHCP

2- Mise en place du serveur WEB

3- Mise en place du serveur DNS

4- Configuration du DNS Reverse

5- Mise en place du blocage DNS

Avant de commencer les manipulations, on fait toujours la commande « **sudo apt update** » pour mettre à jour la machine virtuelle que nous utilisons pour le tp . (Ubuntu ici)

Il faut aussi configurer une adresse ip statique qui se situe dans le LAN et qui correspondra à l'adresse de notre domaine. Ici je choisis « 172.30.6.3/24 » car c'est celle correspondant à mon poste. C'est cette adresse qui sera utilisée dans le WAN (10.10.0.0) pour la résolution de domaine . L'adresse de la passerelle est 172.30.6.254 .

1- Mise en place du DHCP

Nous allons maintenant commencer le TP par la mise en place d'un DHCP sur le LAN de mon poste.

Pour ce faire, il suffit d'utiliser isc-dhcp-server qui est un service utilisé sur la plupart des distributions Linux permettant d'attribuer automatiquement des adresses IP et des paramètres réseau (passerelle, DNS, etc.) aux machines d'un réseau local.

Nous pouvons l'installer avec cette commande : « **sudo apt install isc-dhcp-server** » .

Une fois installé, il faut sélectionner l'interface réseau sur laquelle nous voulons le serveur DHCP .

Il faut pour cela éditer le fichier « **isc-dhcp-server** » qui se trouve dans /etc/default/ comme ci dessous :

```
GNU nano 7.2 /etc/default/isc-dhcp-server *
# Defaults for isc-dhcp-server (sourced by /etc/init.d/isc-dhcp-server)

# Path to dhcpd's config file (default: /etc/dhcp/dhcpd.conf).
#DHCPDv4_CONF=/etc/dhcp/dhcpd.conf
#DHCPDv6_CONF=/etc/dhcp/dhcpd6.conf

# Path to dhcpd's PID file (default: /var/run/dhcpd.pid).
#DHCPDv4_PID=/var/run/dhcpd.pid
#DHCPDv6_PID=/var/run/dhcpd6.pid

# Additional options to start dhcpd with.
# Don't use options -cf or -pf here; use DHCPD_CONF/ DHCPD_PID instead
#OPTIONS=""

# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
# Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACESv4="eth1"
INTERFACESv6=""
```

puis dans « INTERFACESv4 » mettre le nom de l'interface utilisé . Ici il n'y a que une interface qui correspond au lan ou nous voulons le DHCP et son nom est « eth1 »

Après avoir fait cela il va falloir modifier le fichier de configuration du serveur DHCP qui à pour chemin « /etc/dhcp/dhcpd.conf » pour définir le nom de domaine que l'on veut donner, l'adresse du DNS, la plage d'adresse ip à attribuer ainsi que la passerelle du réseau .

```
GNU nano 7.2 /etc/dhcp/dhcpd.conf *
# dhcpd.conf
#
# Sample configuration file for ISC dhcpd
#
# Attention: If /etc/ltsp/dhcpd.conf exists, that will be used as
# configuration file instead of this file.
#
# option definitions common to all supported networks...
option domain-name "leomtrl.ac-monge.fr";
option domain-name-servers 172.30.6.1;

default-lease-time 600;
max-lease-time 7200;
authoritative ;
# The ddns-updates-style parameter controls whether or not the server will
# attempt to do a DNS update when a lease is confirmed. We default to the
# behavior of the version 2 packages ('none', since DHCP v2 didn't
# have support for DDNS.)
ddns-update-style none;
```

Les deux premières lignes « option domain-name » et « option domain-name-servers » correspondent au nom du DNS et à l'adresse ip de celui ci . Sur la capture d'écran on voit 172.30.6.1 mais c'est bien 172.30.6.3 qui correspond à l'adresse ip fixe du poste . La capture d'écran à été prise avant que la modification soit faite .

Ensuite un peu plus bas dans le fichier on peut définir la plage d'adresse à attribuer ainsi que définir la passerelle du réseau qui est celle du lan comme ceci :

```
GNU nano 7.2 /etc/dhcp/dhcpd.conf
#log-facility local7;

# No service will be given on this subnet, but declaring it helps the
# DHCP server to understand the network topology.

#subnet 10.152.187.0 netmask 255.255.255.0 {
#}

# This is a very basic subnet declaration.

subnet 172.30.6.0 netmask 255.255.255.0 {
    range 172.30.6.100 172.30.42.200;
    option routers 172.30.6.254;
    option broadcast-address 172.30.6.255;
}
```

On enregistre le fichier de configuration puis pour que les changements prennent effet, on relance le serveur puis on peut aussi l'activer au démarrage avec ces deux commande :

```
admin1@admin1-Virtual-Machine:~$ sudo systemctl restart isc-dhcp-server
admin1@admin1-Virtual-Machine:~$ sudo systemctl enable isc-dhcp-server
Synchronizing state of isc-dhcp-server.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable isc-dhcp-server
```

Nous avons maintenant mis en place le serveur DHCP sur le LAN de notre poste, nous pouvons passer à la suite avec le serveur WEB .

2- Mise en place du serveur WEB

Il faut donc mettre en place un serveur web afin de pouvoir héberger une page web accessible via le nom de domaine . Pour faire cela, nous allons utiliser Apache car c'est la solution la plus connue et simple .

Nous pouvons l'installer via cette commande : « **sudo apt install apache2** » .

Une fois installé, il faut créer notre page html « vitrine » qui va donc s'afficher lors d'une tentative d'accès réussit au serveur web via le nom de domaine . On peut donc directement modifier le fichier « index.html » qui se trouve dans « var/www/index.html » qui est la page par défaut de apache . On à juste à tout supprimer et à remplacer par notre page à nous comme ceci :

```
GNU nano 7.2 /var/www/html/index.html *
<!doctype html>
<html>
  <head><meta charset="utf-8"><title>Vitrine léo</title></head>
  <body>
    <h1>Vitrine léo</h1>
    <p>Nom : Montreuil Léo</p>
    
  </body>
</html>
```

Une fois la page html faite, il faut créer le répertoire ou l'on va mettre la page web dédié au domaine car la laisser dans le répertoire par défaut peut créer des confusions . On fait donc cette commande :

```
admin1@admin1-Virtual-Machine:~$ sudo mkdir -p /var/www/leomtrl.ac-monge.fr
```

puis pour déplacer la page html vers le nouveau repertoire créer on fait cette commande :

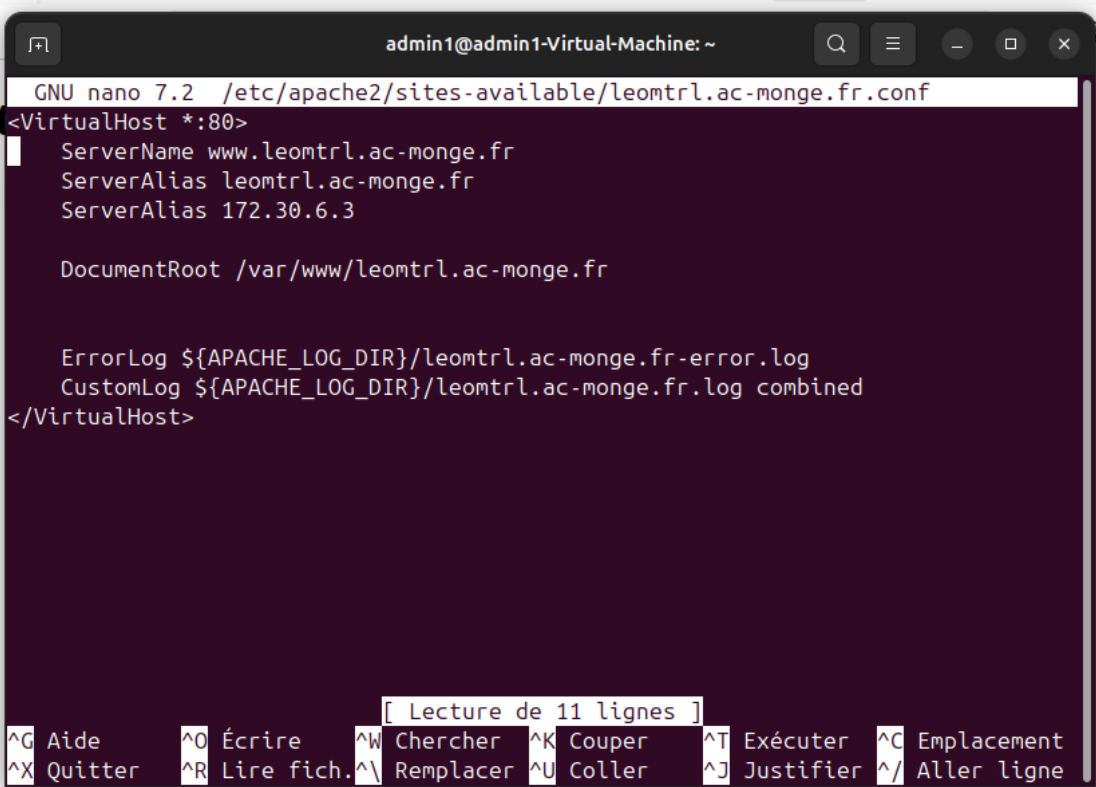
```
admin1@admin1-Virtual-Machine:~$ sudo cp /var/www/html/index.html /var/www/leomtr
rl.ac-monge.fr/
```

le nom du répertoire peut être n'importe lequel. Ici c'est le même nom que celui du domaine (leomtrl.ac-monge.fr) car cela permet simplifier la configuration d'apache et savoir que ce répertoire correspond bien au site du domaine .

Maintenant que la page web est créer et dans le bon répertoire, nous pouvons créer le fichier de configuration VirtualHost d'Apache .c'est le fichier de configuration qui va permettre de lier mon nom de domaine au site web , page web que je veux héberger .

Le but est que quand l'on tape mon nom de domaine sur un navigateur, cela renvoie vers la page html que j'ai crée .

Pour ce faire, on tape la commande « **sudo nano /etc/apache2/sites-available/leomtrl.ac-monge.fr.conf** » pour créer le fichier puis on le configure comme ceci :



```
GNU nano 7.2 /etc/apache2/sites-available/leomtrl.ac-monge.fr.conf
<VirtualHost *:80>
  ServerName www.leomtrl.ac-monge.fr
  ServerAlias leomtrl.ac-monge.fr
  ServerAlias 172.30.6.3

  DocumentRoot /var/www/leomtrl.ac-monge.fr

  ErrorLog ${APACHE_LOG_DIR}/leomtrl.ac-monge.fr-error.log
  CustomLog ${APACHE_LOG_DIR}/leomtrl.ac-monge.fr.log combined
</VirtualHost>
```

[Lecture de 11 lignes]

^G Aide	^O Écrire	^W Chercher	^K Couper	^T Exécuter	^C Emplacement
^X Quitter	^R Lire fich.	^L Remplacer	^U Coller	^J Justifier	^_ Aller ligne

Comme on peut le voir sur la capture d'écran ci dessus, c'est à l'intérieur de ce fichier qu'il faut définir le nom du site web avec « ServerName ».

« ServerAlias » est un nom secondaire du site que l'on peut aussi taper pour accéder à la page WEB . Ici ça permet d'accéder à la page web juste en tapant le nom de domaine sans www au début. Le « ServerAlias 172.30.6.3 » est inutile ici car même sans cette ligne, nous pouvons accéder à la page web via l'ip.

La ligne « DocumentRoot » correspond au chemin où le serveur web doit aller chercher la page html « vitrine » quand quelqu'un veut accéder au site .
Les deux dernières lignes pour les logs ne sont pas nécessaires ici .

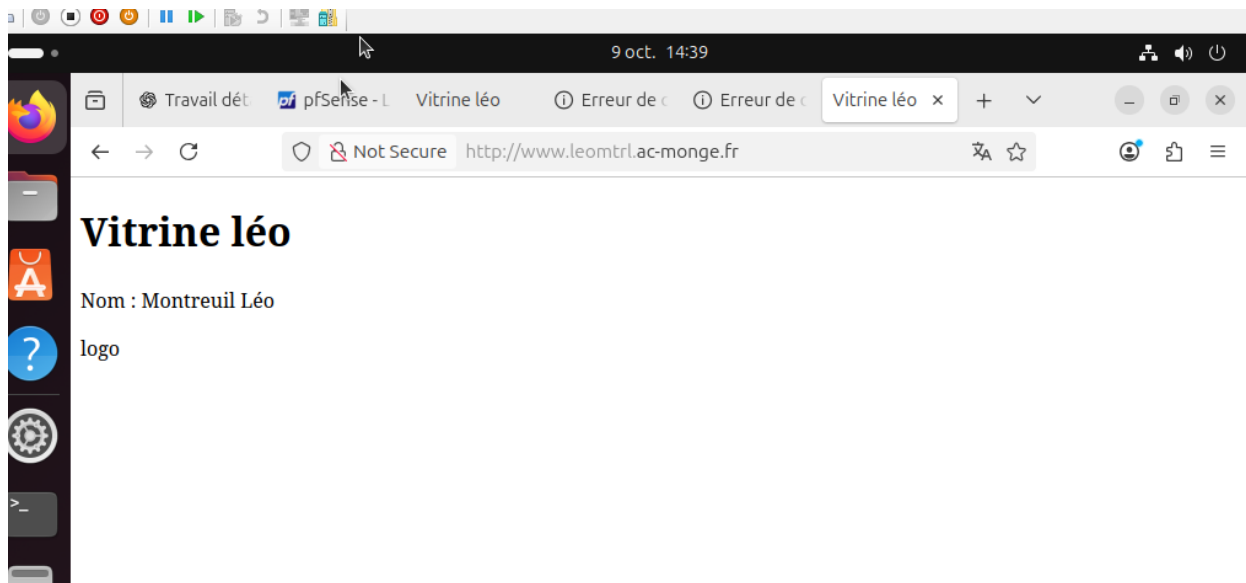
Il ne reste plus qu'à enregistrer le fichier puis à activer notre site web avec la commande ci dessous :

```
admin1@admin1-Virtual-Machine:~$ sudo a2ensite leomtrl.ac-monge.fr.conf
```

Le site est maintenant activé, on redémarre apache avec la commande : « **sudo systemctl restart apache2** » pour que cela soit pris en compte et que le site soit bien actif et accessible .

Nous pouvons maintenant tester si le site est bien accessible via son nom « leomtrl.ac-monge.fr » en local .

Il suffit de taper le nom du site sur un navigateur de la même machine et la page html s'affiche bien .



Ceci est un petit test mais le plus important est de savoir si la page web html est accessible sur tout le WAN via son adresse ip. On doit pouvoir accéder à la page web en tapant 172.30.6.3 sur le navigateur de n'importe quel autre poste qui à un LAN.

Ici je n'ai pas de capture d'écran qui prouve qu'à ce stade la page web est accessible sur tout le LAN mais c'est le cas .

La site web est donc maintenant accessible via son ip mais le but est qu'il soit accessible via le nom de domaine sur tout les autres postes LAN . Ce n'est maintenant pas possible car nous n'avons pas encore mis en place le serveur DNS . Il est donc impossible d'accéder à la page web sur un autre poste juste avec son nom.

C'est pourquoi nous allons maintenant passer à la mise en place du serveur BIND 9 .

3- Mise en place du serveur DNS

La serveur WEB fonctionne, le dhcp est mis en place , nous pouvons donc maintenant mettre en place le serveur DNS .

il va falloir installer Bind9 qui est un serveur DNS permettant de faire la correspondance entre un nom de domaine et une adresse IP . Bind9 va donc nous servir à héberger notre domaine (leomtrl.ac-monge.fr), à associer le nom de notre site web à notre adresse ip, à réaliser la résolution inverse (DNS Reverse) et à bloquer certains sites (blocage DNS)

Pour l'installer nous allons donc utiliser cette commande :

```
admin1@admin1-Virtual-Machine:~$ sudo apt install -y bind9 bind9utils
Lecture des listes de paquets... Fait
```

Une fois installé, il faut créer le dossier « zones » figurera ensuite notre zone que nous allons créer . Une zone dans bind9 est un fichier de configuration DNS qui contient toutes les informations sur un

domaine précis . C'est dans ce fichier que l'on définit toutes les correspondances entre nom et ip pour notre domaine .

On fait donc cette commande :

```
admin1@admin1-Virtual-Machine:~$ sudo mkdir -p /etc/bind/zones
```

ensuite, on va créer notre zone qui est notre fichier de configuration DNS dans ce dossier puis on va le configurer comme ceci :

```
GNU nano 7.2 /etc/bind/zones/db.leomtrl.ac-monge.fr
$TTL 3600
@ IN SOA ns.leomtrl.ac-monge.fr. admin.leomtrl.ac-monge.fr. (
    2025100801 ; serial YYYYMMDDnn
    3600      ; refresh
    1800      ; retry
    604800    ; expire
    3600 )    ; negative cache

IN NS ns.leomtrl.ac-monge.fr.

ns IN A 172.30.6.3 ;
www IN A 172.30.6.3 ;
@ IN A 172.30.6.3
```

La deuxième ligne correspond au serveur DNS maître de la zone, celui qui fait autorité sur le domaine, ici leomtrl.ac-monge.fr (ns=nameserver) .

La ligne « IN NS » correspond au serveur responsable du domaine. Ensuite les ligne suivante « IN A » servent à faire l'association entre l'adresse ip et le serveur DNS et site WEB .

Une fois configuré, on peut donc enregistrer le fichier et il faut maintenant déclarer la zone dans le fichier de configuration local pour qu'elle soit prise en compte .

Il faut alors accéder au fichier qui se trouve au chemin «/etc/bind/named.conf.local » puis ajouter sa zone comme ci dessous :

```
admin1@admin1-Virtual-Machine: ~
GNU nano 7.2 /etc/bind/named.conf.local *
//
// Do any local configuration here
//
zone "leomtrl.ac-monge.fr" {
    type master;
    file "/etc/bind/zones/db.leomtrl.ac-monge.fr";
    allow-update { none; };
};

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";
```

Maintenant que la zone est déclaré, on enregistre et on peut passer au forwarding .

Le forwarding est ce qui va permettre à notre DNS d'envoyer les requêtes qu'il ne connaît pas vers un autre serveur DNS . Le but ici est d'envoyer les requêtes inconnu vers le serveur racine, principale (10.10.0.1) pour que lui puisse trouver la réponse aux requêtes et nous la renvoyer. Cela est important car notre DNS connaît uniquement le domaine local qui est le notre, pour les requêtes extérieur au domaine local il faut donc demander à un autre DNS .

Pour configurer le forwarding il suffit d'accéder au fichier « named.conf.options » puis y indiquer l'adresse du domaine ou l'on veut rediriger les requêtes comme ceci :

```
GNU nano 7.2 /etc/bind/named.conf.options *
// Uncomment the following block, and insert the addresses replacing
// the all-0's placeholder.

// forwarders {
//     10.10.0.1;
// };

//=====
// If BIND logs error messages about the root key being expired,
// you will need to update your keys.  See https://www.isc.org/bind-k
//=====
dnssec-validation auto;

listen-on { any; };
listen-on-v6 { any; };

allow-query { any; };
recursion yes;
};
```

Il faut bien penser à retirer les deux barres avant les lignes pour le forwarders car aussi non les changements ne seront pas pris en compte .

Une fois cela fait, le forwarding est configuré, on peut enregistrer .

Il ne reste plus qu'à vérifier la syntaxe des fichiers de configuration pour savoir si le DNS est prêt à être utilisé . Pour ce faire, on utilise ces deux commandes fournit dans BIND9 :

```
root@debian:~# sudo named-checkconf
sudo named-checkzone leomtrl.ac-monge.fr /etc/bind/zones/db.leomtrl.ac-monge.fr
zone leomtrl.ac-monge.fr/IN: loaded serial 2025100801
OK
root@debian:~#
```

Utiliser mot de passe sudo

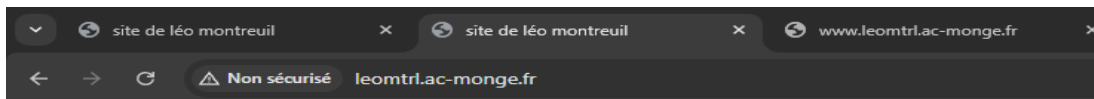
À Teste DNS

On peut voir le statut «OK », cela signifie que le DNS est prêt et qu'il n'y a pas d'erreur .

Pour être sûr que le DNS soit pris en compte ; on redémarre BIND 9 avec la commande « **sudo systemctl restart BIND9** » et « **sudo systemctl restart named** »

Maintenant que notre serveur DNS est mis en place, on va pouvoir tester si notre DNS fonctionne en essayant d'accéder à notre page web via notre nom de domaine sur un autre poste LAN .

On tape notre nom de domaine sur le navigateur d'un autre poste et voici ce que ça donne :



Bienvenue sur le site de leo montreuil

Nom : Batman

la page web est bien accessible juste avec le nom de domaine sur un autre poste (la page n'est plus la même, je l'ai changé entre temps) .

Pour être encore plus sûr du bon fonctionnement du dns, on peut utiliser la commande dig qui est un outil permettant d'interroger un nom de domaine pour savoir qu'elle ip correspond à celui-ci .

```
root@debian:~# dig www.leomtrl.ac-monge.fr

; <<>> DiG 9.18.33-1~deb12u2-Debian <<>> www.leomtrl.ac-monge.fr
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 57203
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1432
;; QUESTION SECTION:
;www.leomtrl.ac-monge.fr.      IN      A

;; ANSWER SECTION:
www.leomtrl.ac-monge.fr. 3212    IN      A      172.30.6.3

;; Query time: 4 msec
;; SERVER: 10.10.0.1#53(10.10.0.1) (UDP)
;; WHEN: Wed Oct 15 11:08:42 CEST 2025
;; MSG SIZE rcvd: 68

root@debian:~#
```

Il faut aussi vérifier que le forwarding fonctionne, on peut donc utiliser l'outil nslookup pour vérifier que quand l'on fait une requête inconnue, celle-ci est renvoyée vers le serveur 10.10.0.1. Voici ce que ça donne quand on envoie une requête en utilisant le dns 172.30.6.3 vers un autre DNS inconnu :

```
> cerise.ac-monge.fr
Server:          172.30.6.3
Address:         172.30.6.3#53

Non-authoritative answer:
Name:   cerise.ac-monge.fr
Address: 172.30.5.3
>
```

le serveur à envoyer la requête vers le 10.10.0.1 qui a trouvé l'adresse du DNS (172.30.5.3) et qui nous l'a renvoyé.

Pareil quand j'essaie directement d'envoyer une requête vers le serveur racine 10.10.0.1 :

```
> ac-monge.fr
Server:          172.30.6.3
Address:         172.30.6.3#53

Non-authoritative answer:
Name:   ac-monge.fr
Address: 213.186.33.5
>
```

4- Configuration du DNS Reverse

Le DNS est donc fonctionnel, le forwarding aussi, il va falloir maintenant faire le DNS Reverse pour que quand l'on interroge l'adresse du domaine (172.30.6.3), cela renvoie vers le nom de domaine.

Il va falloir faire la même chose que pour la résolution normale, c'est à dire créer une zone mais qui va être la zone inverse et la déclarer.

On crée donc le fichier de configuration de la zone inverse comme ceci :

```
GNU nano 7.2 /etc/bind/zones/db.6.30.172 *
$TTL 3600
@ IN SOA ns.leomtrl.ac-monge.fr. admin.leomtrl.ac-monge.fr. (
    2025101501 ; serial
    3600      ; refresh
    1800      ; retry
    604800    ; expire
    3600 )    ; minimum

IN NS ns.leomtrl.ac-monge.fr.

3 IN PTR www.leomtrl.ac-monge.fr.
```

Au lieu d'appeler la zone par le nom de domaine, on l'appelle par la zone inverse c'est à dire qu'on écrit les octets en ordre inversé (6.30.172).

On déclare ensuite la zone dans le fichier dédié :

```
admin@debian: ~
GNU nano 7.2 /etc/bind/named.conf.local *
zone "leomtrl.ac-monge.fr" {
    type master;
    file "/etc/bind/zones/db.leomtrl.ac-monge.fr";
    allow-transfer { none; };
};
zone "6.30.172.in-addr.arpa" {
    type master;
    file "/etc/bind/zones/db.6.30.172";
    allow-update { none; };
};
```

on doit rajouter le suffixe « in-addr.arpa » à la fin de la zone pour montré que c'est une résolution inverse. C'est un suffixe qui est dédié à ça .

Ensuite on enregistre et on vérifie la syntaxe avec les même commande qu'auparavant :

```
root@debian:~# sudo named-checkconf
sudo named-checkzone 6.30.172.in-addr.arpa /etc/bind/zones/db.6.30.172
zone 6.30.172.in-addr.arpa/IN: loaded serial 2025101501
OK
root@debian:~#
```

puis on redémarre bind 9 et on teste d'envoyer une requête vers l'adresse du serveur pour savoir si cela répond le nom de domaine .

```
root@debian:~# sudo systemctl restart bind9

root@debian:~# nslookup
> 172.30.6.3
3.6.30.172.in-addr.arpa name = www.leomtrl.ac-monge.fr.
>
```

On peut voir que la résolution inverse fonctionne .

5- Mise en place du blocage DNS

Pour terminer ce TP , Il ne reste plus qu'à faire le blocage DNS pour empêcher l'accès au site leboncoin.fr et amazon.uk .

Encore la même chose que pour la résolution normal et inverse, il faut créer une zone dédié au sites que l'on veut bloquer puis renvoyer les requêtes vers une erreur ou vers le localhost .

On va donc appeler cette zone « block » et configurer comme ceci :

```
GNU nano 7.2 /etc/bind/zones/db.block *
```

```
$TTL 3600
@ IN SOA localhost. root.localhost. (
    2025101501 ;
    3600      ; refresh
    1800      ; retry
    604800    ; expire
    3600 )    ; minimum

IN NS localhost.
* IN A 127.0.0.1
```

Tout les sites qui seront rattaché à cette zones renverront vers le localhost, ne seront pas accessibles .

Ensuite, on déclare la zone avec les sites que l'on veut bloquer dans le fichier dédié :

```
GNU nano 7.2 /etc/bind/named.conf.local *
zone "leomtrl.ac-monge.fr" {
    type master;
    file "/etc/bind/zones/db.leomtrl.ac-monge.fr";
    allow-transfer { none; };
};
zone "6.30.172.in-addr.arpa" {
    type master;
    file "/etc/bind/zones/db.6.30.172";
    allow-update { none; };
};
zone "leboncoin.fr" {
    type master;
    file "/etc/bind/zones/db.block";
};
zone "amazon.uk" {
    type master;
    file "/etc/bind/zones/db.block";
};
```

on peut voir que le domaine amazon.uk et leboncoin.fr sont associé à la zone « block » .

Après avoir fait cela, on vérifie les syntaxes puis on redémarre bind9 et on teste d'accéder aux site pour voir si le blocage fonctionne .



Bienvenue sur le site de batman montreuil

On peut voir qu'en tentant d'accéder à leboncoin.fr, cela redirige vers ma page web, le localhost .

Difficultés rencontrés : j'ai essayé de faire le TP en utilisant NGINX au premier abord mais je n'arrivais pas à accéder à ma page web sur d'autres postes alors que j'étais allé jusqu'au DNS . J'ai du tout recommencer à zéro avec Apache sur une autre machine virtuelle et cela a ensuite fonctionné .

C'est pour cela qu'il y a des captures d'écran de deux machines différentes dans le compte rendu .

Sources utilisées : ChatGPT .